

LogMend

Context-Aware Log Anomaly Detection & TTP Analysis

Complete Project Reference — Workflow · Tools · ML Ensembles

Project overview

LogMend is a dual-dataset security log anomaly detection system built on the LogRESP-Agent framework. It combines a Neo4j provenance graph, a 7-tool modular agent, and a recursive Thought–Action–Observation (TAO) reasoning loop driven by a large language model. The system operates on two complementary datasets: **BETH** (honeypot intrusion logs — botnet, SSH brute-force, cryptomining) and **DARPA THEIA** (APT kill chains — Firefox exploit, kernel rootkit, process injection, C2 communication). Detection is performed by separate dataset-specific ML ensembles whose output is fused inside the SequenceScorer tool.

Dataset architecture and Neo4j ingestion strategy

Split	BETH	THEIA (DARPA E5)	Purpose
Training	763,144 records 8 hosts (benign only) Feb. data CSV	All pre-attack JSON CDM (May 7–13 window) AVRO ingested	Neo4j graph build. Only training split is ingested. Forms the benign baseline.
Validation	188,967 records 4 hosts (benign only) Feb. data CSV	May 14 mixed period (30% malicious subjects)	Hyperparameter tuning. Not in Neo4j graph. Raw log evaluation only.
Testing	188,967 records 1 host (attack present) Feb. data CSV	May 14–17 attack window (50% malicious subjects)	Final evaluation. Not in Neo4j graph. End-to-end pipeline test.

Neo4j node types: Process · FileObject · NetFlowObject · Principal (User) | **Relationships:** SPAWNED · ACCESSED · CONNECTED_TO · RUNS_AS | **BETH key:** PID + hostName | **DARPA key:** UUID

ML ensemble compositions — final confirmed

BETH ensemble — GraphCL + Isolation Forest + SAD			
Model	Role	Tier	Weight
GraphCL	Self-supervised GNN contrastive learning on benign graph topology	Standard GNN	40%
Isolation Forest	Unsupervised tabular anomaly isolation — no labels required	Standard ML	30%
SAD	Semi-supervised graph anomaly detection — uses sus=1 labels in graph space	Research-grade	30%

DARPA THEIA ensemble — MAGIC + Flash + ORTHRUS			
Model	Role	Tier	Weight

MAGIC	Hybrid contrastive loss, neighborhood-level APT detection on provenance graphs. No CDM schema version dependency — E5 native.	Tier 4 (USENIX Sec 2024)	40%
Flash	Provenance graph repr. with semantic node encoding. Handles sparse attributes via graph structure — robust to THEIA/TRACE 54–76% missing attribute rate.	Tier 4 (IEEE S&P; 2024)	35%
ORTHRUS	High-quality attribution IDS — eliminates dependency explosion false positives. Works on graph structure alone.	Tier 5 (USENIX Sec 2025)	25%

Why MAGIC + Flash replace ThreaTrace + NodLink: ThreaTrace ships only with E3 (CDM v18) models and achieves F1=0.00 on TRACE and F1=0.44 on THEIA without schema migration. MAGIC (USENIX Sec 2024) uses neighborhood-level contrastive loss on provenance graphs and carries no CDM version dependency — it is trained on your E5 data directly. NodLink's semantic VAE degrades severely at 54–76% missing attributes; Flash (IEEE S&P; 2024) encodes provenance semantics through graph structure rather than attribute completeness, making it robust to the same missing-attribute rates. Both are Tier 4 — proven on DARPA TC data of the same lineage.

Full LogMend workflow — start to end

1. Offline — Training data ingestion (Neo4j graph build)

BETH training CSVs (8 hosts, 763K records) and THEIA pre-attack JSON CDM files (May 7–13, benign window) are parsed and loaded into Neo4j as a provenance graph. Nodes: Process, FileObject, NetFlowObject, Principal. Edges: SPAWNED, ACCESSED, CONNECTED_TO, RUNS_AS. BETH nodes keyed by PID+hostName; DARPA by UUID. This step runs once and produces the baseline graph. Validation and test splits are NOT ingested — they remain as raw log files.



2. Offline — ML ensemble training

BETH: GraphCL trains on the benign provenance subgraph using contrastive augmentation (edge dropout + feature masking). Isolation Forest trains on 9-feature tabular vectors. SAD trains with sus=1 positives in graph space. DARPA: MAGIC trains neighbourhood-level contrastive embeddings on THEIA/TRACE provenance graphs. Flash trains semantic node encodings. ORTHRUS trains its attribution IDS. All models consume the 9-feature vector: is_root, event_count, sus_ratio, evil_ratio, bias, net_ratio, c2_event_count, file_write_count, inject_count.



3. Online — Raw log arrival (validation / test)

A new raw log record arrives — either a BETH CSV row or a THEIA CDM JSON record. The record has not been seen during training. This is the entry point for the live detection pipeline.



4. T1 — LogParser (parse raw record)

The LogParser (base LogRESP-Agent design) reads the raw log and standardises it into a unified dict: {processName, pid, hostName, uuid, parentPid, commandLine, userId, eventType, timestamp, datasetTag}. BETH records are parsed from CSV fields; THEIA records from CDM JSON. The datasetTag ('beth' | 'darpa') is injected here for all downstream routing. No Neo4j query at this stage.



5. LLM Planner — first Thought cycle (TAO loop begins)

The standardised record is handed to the LLM Planner (Claude Sonnet via Anthropic API). The planner generates a Thought (hypothesis about the event) and selects the most relevant tool to invoke next, using its internal utility function over tool relevance and expected information gain.



6. T3 — DescriptionGenerator

Usually the first tool invoked. Converts the parsed record into a natural language description sentence. For DARPA records, an optional Neo4j query via CONNECTED_TO pulls remote addresses and C2 flags to enrich the description. Output feeds directly into the LLM planner's next reasoning cycle.



7. T2 — RuleMatcher

Applies 12 pattern signatures to the record: 6 general Linux attack patterns and 6 DARPA APT-specific patterns (kernel module load, ptrace sshd, drive-by C2, Firefox→shell chain, sshdlog persistence, known C2 IPs). Returns matched rule IDs.



8. T4 — SequenceScorer (ML ensemble routing + scoring)

Computes the 9-feature vector from the record. Reads the datasetTag: 'beth' → BETH ensemble (GraphCL 40% + IF 30% + SAD 30%); 'darpa' → DARPA ensemble (MAGIC 40% + Flash 35% + ORTHRUS 25%). Returns weighted anomaly score [0,1] and individual model scores. Score thresholds: <0.35 benign, 0.35–0.65 suspicious, >0.65 malicious.



9. T5 — ContextRetriever (if anomaly score > threshold)

Queries Neo4j for the process neighbourhood: parent/child processes via SPAWNED, accessed files via ACCESSED, network connections via CONNECTED_TO. Returns process lineage depth, file access list, remote IP list, and has_c2_connection boolean. Provides the TAO loop with graph context to enrich the hypothesis.



10. T6 — TTPMapper

Embeds the description with MiniLM-L12-v2 and computes cosine similarity against 11 stored TTP embeddings (5 Linux general + 6 DARPA APT-specific). Returns top-matched TTP ID, name, tactic, and similarity score.



11. T7 — ThreatLookup

Looks up TTP IDs or C2 IP addresses in the knowledge base. Returns human-readable descriptions, tactic mappings, and (for C2 IPs) threat actor attribution (Drakon / Azazel APT). Enriches the agent's final explanation.



12. TAO loop — continue or conclude

The planner evaluates accumulated observations. If confidence < threshold, it re-enters the loop selecting another tool. If confidence >= threshold (malicious or benign with high certainty), the loop terminates.



13. Final explanation output

The planner synthesises all observations into a structured report: (a) Verdict — benign / suspicious / malicious; (b) Evidence — key observations from each tool; (c) TTP map — matched MITRE ATT&CK; IDs and tactics; (d) Reasoning trace — ordered sequence of tool invocations; (e) Confidence score.

Tool descriptions — properties and features

T1 — LogParser

BETH + DARPA

The LogParser is the pipeline entry point — it takes a raw incoming log record (a BETH CSV row or a THEIA CDM JSON event) and standardises it into a unified Python dict consumed by all downstream tools. This is the LogRESP base-paper design: a pure parser, not a graph query. No Neo4j interaction occurs here.

Key properties:

- Accepts BETH CSV format: processId, hostName, parentProcessId, processName, userId, eventType, stackAddresses, sus, evil
- Accepts THEIA CDM JSON format: uuid, type, cmdLine, ppid, startedBy (Subject records)
- Injects datasetTag = 'beth' or 'darpa' based on input format detection
- Normalises field names to unified schema: {processName, pid, hostName, uuid, parentPid, commandLine, userId, eventType, timestamp, datasetTag}
- Returns None with logged warning for malformed or missing records
- Stateless — no memory between calls; idempotent on identical input

DARPA-specific additions:

- [uuid field passthrough for DARPA CDM records](#)
- [datasetTag injection — drives all downstream routing decisions](#)
- [CDM JSON Subject record parser \(type, cmdLine, ppid, startedBy fields\)](#)

T2 — RuleMatcher

BETH + DARPA

Applies a library of 12 pattern signatures against the standardised record to detect known malicious behaviours. The original 6 Linux patterns are retained unchanged. Six new APT-specific patterns derived from the THEIA/TRACE E5 attack ground truth are added. A hardcoded C2 IP blocklist is also embedded.

Key properties:

- Pattern library: 12 signatures total stored as compiled regex + string matchers
- Linux patterns (6): reverse shell, data exfiltration, chmod 777, /etc/shadow access, base64 pipe decode, /tmp/ execution
- DARPA APT patterns (6): suspicious .ko insmod from non-root, ptrace on sshd, outbound non-standard port from browser, Firefox→network-proc→shell chain, sshdlog persistence artifact, known C2 IP match
- C2 IP blocklist: 35.106.122.76, 69.155.209.87, 189.141.204.211, 208.203.20.42
- Returns: {matched_rules: [rule_ids], severity: 'none'|'low'|'high', c2_match: bool}
- Severity: high if ≥1 APT pattern or C2 IP match; low if ≥1 Linux pattern; else none

DARPA-specific additions:

- [6 APT-specific pattern signatures](#)
- [C2 IP blocklist embedded as a frozen set for O\(1\) lookup](#)
- [c2_match boolean in return dict](#)

T3 — DescriptionGenerator

BETH + DARPA

Converts a standardised log record into a rich natural language sentence that the LLM planner can reason over directly. Previously generated from process name, parent, and command line only. Now includes network connection context for DARPA records via an optional Neo4j join through CONNECTED_TO.

Key properties:

- Base sentence template: '{processName} (pid={pid}) spawned by {parent} executed: {commandLine} as userId={userId}'
- For DARPA records: appends network context — 'connected to {remoteAddr}:{port}'
- C2 annotation: appends '(known C2 — Drakon/Azazel APT)' when is_c2=true
- Spawn chain depth: 'process depth={n} in lineage' for deep spawning
- Falls back gracefully when commandLine is None/empty (common in THEIA/TRACE)
- Returns a single plain-text string, ≤ 300 characters, for embedding

DARPA-specific additions:

- OPTIONAL MATCH (p)-[:CONNECTED_TO]->(n:NetFlowObject) for DARPA records
- C2 flag annotation in natural language output
- Spawn chain depth field from Neo4j parent traversal

T4 — SequenceScorer

BETH + DARPA

The core ML scoring engine. Computes a [0,1] anomaly score using a dataset-specific weighted ensemble of trained models. This is the most architecturally significant tool in the pipeline: the feature vector grows from 5 to 9 features, two separate ensemble model sets are maintained, and routing between them is driven by the datasetTag injected by LogParser.

Key properties:

- 9-feature input vector: is_root (bool), event_count (int), sus_ratio (float), evil_ratio (float), bias (const 5.0), net_ratio (float), c2_event_count (int), file_write_count (int), inject_count (int)
- BETH ensemble: GraphCL (40%) + Isolation Forest (30%) + SAD (30%)
- DARPA ensemble: MAGIC (40%) + Flash (35%) + ORTHRUS (25%)
- LogGraphSAGE in_channels=9 (updated from 5 in both sequence_scorer.py and train_true_gnn.py)
- Routing: reads datasetTag from record — 'beth' or 'darpa'
- Thresholds: <0.35 = benign, 0.35–0.65 = suspicious, >0.65 = malicious
- Returns: {score: float, verdict: str, model_scores: dict, dataset: str}

DARPA-specific additions:

- 4 new features: net_ratio, c2_event_count, file_write_count, inject_count
- DARPA ensemble: MAGIC + Flash + ORTHRUS model set
- Dataset routing logic on datasetTag
- in_channels: 5 → 9 architecture update

T5 — ContextRetriever

BETH + DARPA

Expands the investigation context by querying Neo4j for the process neighbourhood. Three traversal types are supported: parent/child spawning lineage, file accesses, and (new) network connections. The network traversal adds C2 flagging directly into the context dict, which drives the TAO loop toward TTPMapper when C2 activity is present.

Key properties:

- Traversal 1 — SPAWNED: MATCH (p)-[:SPAWNED*1..3]->(child) — up to 3 hops
- Traversal 2 — ACCESSED: OPTIONAL MATCH (p)-[:ACCESSED]->(f:FileObject)
- Traversal 3 — CONNECTED_TO: OPTIONAL MATCH (p)-[:CONNECTED_TO]->(n:NetFlowObject)
- Returns: {parent_process, child_processes[], accessed_files[], network_connections[], has_c2_connection: bool}
- BETH query uses PID+hostName composite key
- DARPA query uses UUID key
- Returns empty collections (not None) when no neighbours exist

DARPA-specific additions:

- [CONNECTED_TO traversal — third OPTIONAL MATCH clause](#)
- [network_connections list in return dict](#)
- [has_c2_connection boolean flag](#)
- [UUID-keyed query path for DARPA](#)

T6 — TTPMapper

BETH + DARPA

Maps the event description to MITRE ATT&CK; techniques using MiniLM-L12-v2 sentence embeddings and cosine similarity. The original 5 Linux TTPs are retained. Six APT-specific TTPs derived from the THEIA/TRACE E5 attack report are added, covering the full Firefox → kernel exploit → sshd injection → persistence kill chain.

Key properties:

- Embedding model: sentence-transformers/paraphrase-MiniLM-L12-v2 (local inference)
- Similarity metric: cosine similarity, top-1 match returned
- 11 TTPs total: T1059 (cmd exec), T1070 (indicator removal), T1003 (OS cred dump), T1041 (exfiltration C2), T1078 (valid accounts), T1189 (drive-by compromise), T1068 (priv esc exploit), T1055 (process injection), T1547 (boot autostart), T1071 (app layer protocol C2), T1014 (rootkit)
- Returns: {ttp_id, name, tactic, similarity_score}
- Minimum similarity threshold: 0.45 — below this, returns {ttp_id: None}

DARPA-specific additions:

- [T1189 — Drive-by Compromise \(Firefox exploit\)](#)
- [T1068 — Exploitation for Privilege Escalation \(kernel module\)](#)
- [T1055 — Process Injection \(ptrace sshd\)](#)
- [T1547 — Boot/Logon Autostart \(sshdlog persistence\)](#)
- [T1071 — Application Layer Protocol \(C2 over HTTP\)](#)
- [T1014 — Rootkit \(Azazel APT\)](#)

T7 — ThreatLookup

BETH + DARPA

Enriches the final investigation by returning concise human-readable descriptions of known TTPs and threat actors from an in-memory knowledge base dictionary. The smallest change of the seven tools — purely additive. Six new TTP entries and a C2 IP lookup table are appended.

Key properties:

- Knowledge base: Python dict keyed by TTP ID or C2 IP string
- 11 TTP entries: Linux general (5) + DARPA APT-specific (6)
- C2 IP table: 4 known E5 malicious IPs → threat actor attribution
- Input: TTP ID string (e.g. 'T1055') or IP address string
- Returns: {name, tactic, description, threat_actor (if IP)}
- Returns {error: 'not found'} for unknown inputs — never raises

DARPA-specific additions:

→ [T1189](#), [T1068](#), [T1055](#), [T1547](#), [T1071](#), [T1014](#) entries

→ C2 IP lookup: [35.106.122.76](#) / [69.155.209.87](#) / [189.141.204.211](#) / [208.203.20.42](#) → [Drakon/Azazel](#) attribution

TAO loop mechanics and final output format

Stage	What happens	Key decision
Thought (T)	LLM generates hypothesis from description + prior observations	Which tool has highest utility given current evidence?
Action (A)	Selected tool is invoked with standardised record	Tool result appended to short-term memory
Observation (O)	Result is interpreted; confidence re-evaluated	$\text{confidence} \geq \theta \rightarrow \text{terminate}$; else new Thought cycle

Tool selection utility function

At each Thought step the LLM scores every available tool on two dimensions, both rated 1–5: **relevance**(T_j, G) — semantic alignment between the tool's function and the current analysis goal — and **expected_gain**(T_j, O_{1:t-1}) — estimated information gain given what has already been observed. The tool with the highest combined score is invoked. This is evaluated directly by the LLM through contextual reasoning, not by a fixed formula.

Termination condition

The loop terminates when $\text{confidence_malicious}(O) \geq \theta_{\text{mal}} \text{ (default 0.75)}$ or $\text{confidence_benign}(O) \geq \theta_{\text{ben}} \text{ (default 0.80)}$. Confidence is assessed by the LLM across all accumulated observations — it is semantic convergence of multiple signals, not a single numeric threshold.

Final output structure

Field	Content
verdict	'benign' 'suspicious' 'malicious'
confidence	Float [0,1] — overall confidence in verdict
anomaly_score	Weighted ML ensemble score from SequenceScorer
evidence	List of key observations from all invoked tools
matched_rules	RuleMatcher output — list of triggered rule IDs
ttp_mapping	TTPMapper output — MITRE ATT&CK; ID, name, tactic, score
threat_context	ThreatLookup output — TTP description, actor attribution
reasoning_trace	Ordered list of {tool, input, output} dicts per TAO cycle
dataset	'beth' 'darpa' — which ensemble was used